OWL

Fall 2026

Outlier Weighed Layerwise Sparsity on small modern LLMs

CAI 5307 - NLP Project

Tuan Khang Phan
Xuan Gia Han Nguyen

USF UNIVERSITY OF SOUTH FLORIDA

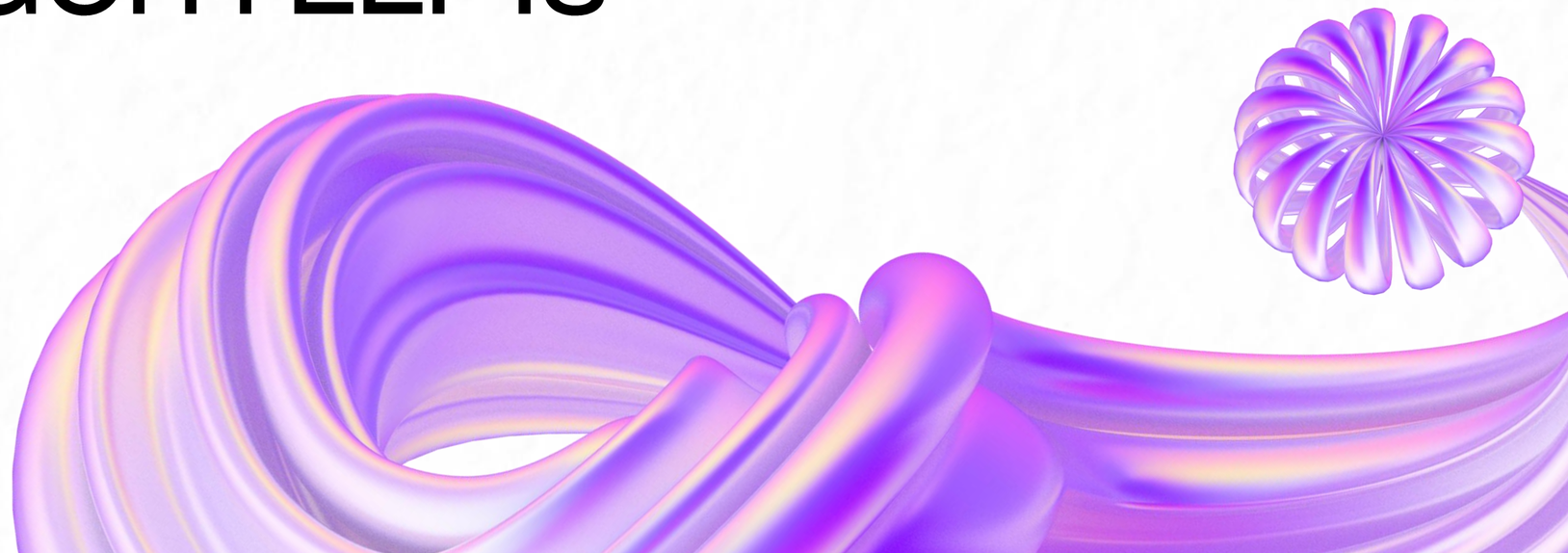


Table of Contents

What does OWL claim?

One-shot pruning and the OWL idea

Can we reproduce it?

Four claims, four experiments

Does it hold on new LLMs?

Our extension to Qwen2.5 and Llama 3

What do we need?

Models, GPU hours, timeline

1. What does OWL claim?

Pruning every layer equally breaks at 70%

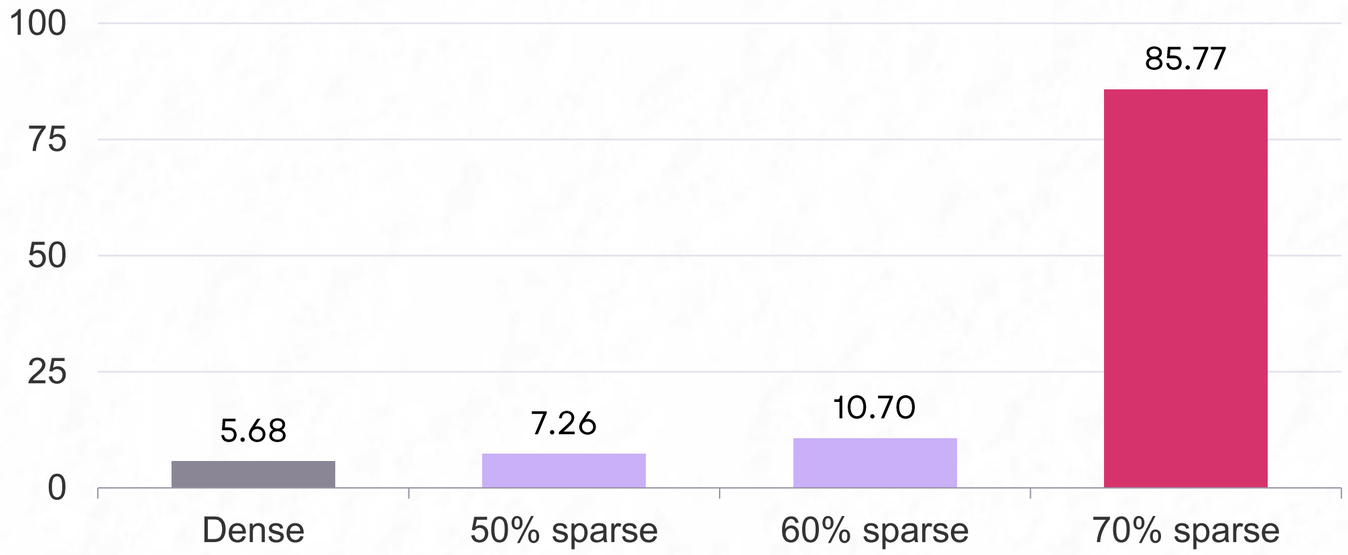
Dense LLM
LLaMA-7B

→

Prune 70% of weights
one shot, no retraining

→

Sparse LLM
same sparsity in every layer



Wanda and SparseGPT score each weight, then cut the same share from every layer.

15x

worse than dense at 70% sparsity

LLaMA-7B pruned with Wanda, uniform sparsity. Lower is better. Yin et al. 2024, Tables 3 and 8.

1. What does OWL claim?

OWL prunes outlier-rich blocks less

1 **Score every weight**

Wanda metric: input norm times weight size

A = ||X|| · |W|

→

2 **Count outliers**

Per block: share of weights scoring over M times the mean

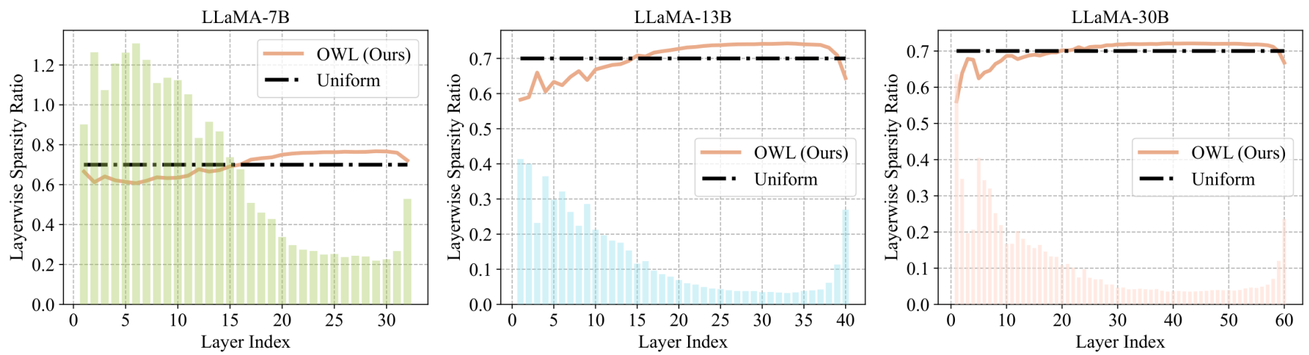
LOD, one value per block

→

3 **Set block sparsity**

More outliers, lower sparsity, inside a narrow band

Block sparsity ~ 1 − LOD



Bars are the outlier share of each block.

Orange line is OWL's sparsity. It dips where the bars are tall.

Dashed line is uniform 70%.

Figure 1 of Yin et al. 2024 (LLaMA-7B, 13B, 30B at 70% sparsity).

1. What does OWL claim?

Four claims we test

C1 **85.77 → 24.55**

OWL beats uniform sparsity at 70% (LLaMA-7B perplexity, Wanda). On OPT-6.7B it hurts SparseGPT.

C2 **61.22 vs 6.62**

The gain grows as models shrink: perplexity drop with Wanda at 7B vs 65B.

C3 **39.4% → 46.5%**

Zero-shot accuracy improves too. Mean of 7 tasks, LLaMA-7B at 70%, Wanda.

C4 **Not uniform**

Outliers concentrate in some blocks (LOD), and good pruning methods keep them.

2. Can we reproduce it?

Four experiments, one per claim

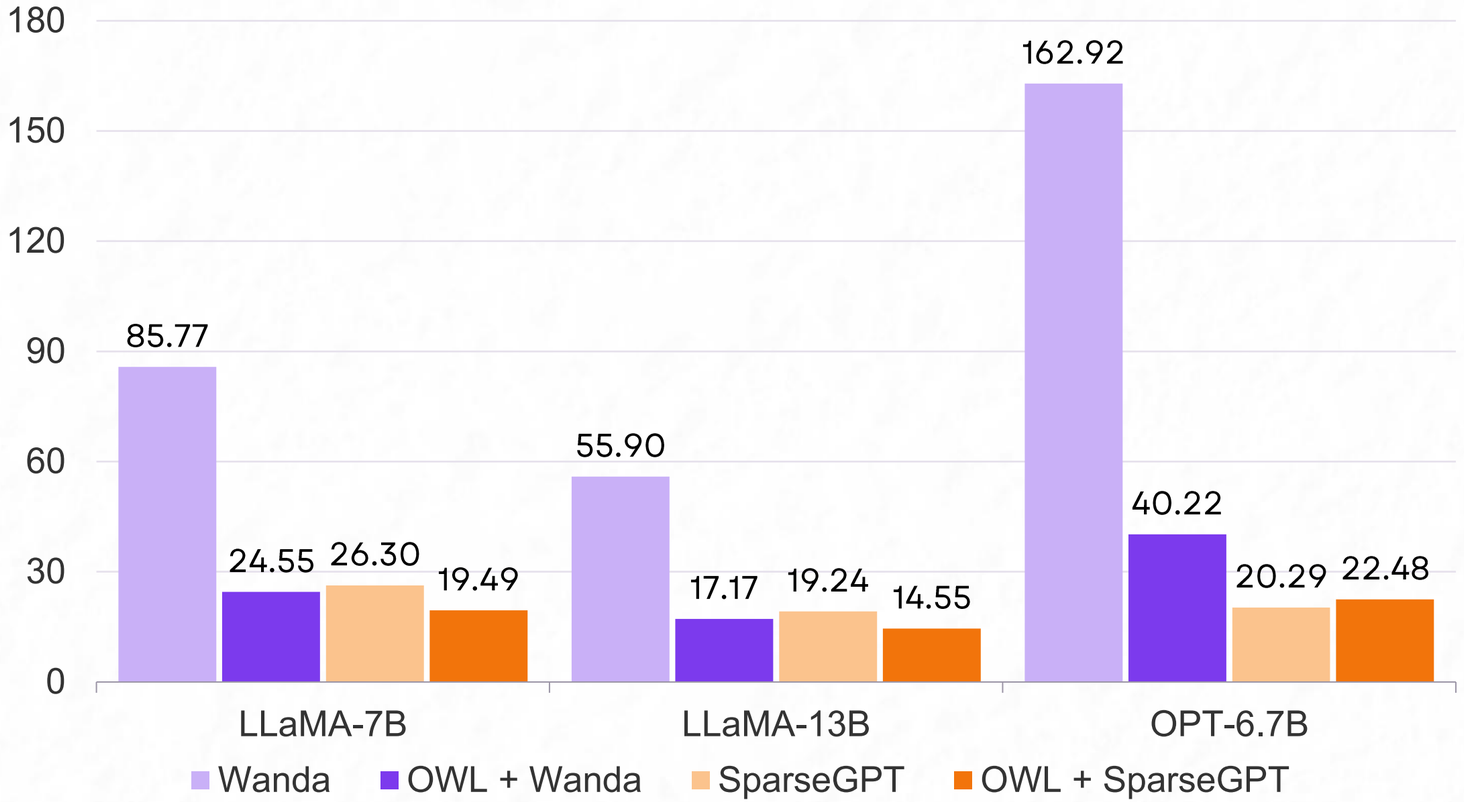
Exp.	Paper target	What we run	Claim
R1	Table 3	70% sparsity on LLaMA-7B, LLaMA-13B and OPT-6.7B. Magnitude, Wanda, SparseGPT, uniform vs OWL	C1, C2
R2	Figure 2, Table 8	40% to 80% sweep on LLaMA-7B with Wanda, six ways to split sparsity	C1
R3	Table 4	Zero-shot accuracy on 7 tasks, dense vs 70% sparse LLaMA-7B	C3
R4	Table 1, Figure 1	Outlier distribution of each dense model, again after pruning	C4

Reproduced means the dense LLaMA-7B perplexity is 5.68, every R1 setting moves in the paper's direction, and our 70% perplexities are within 15% of the paper's.

Left out: LLaMA-30B and 65B, the CPU speedup study, LoRA fine-tuning.

2. Can we reproduce it?

R1: Table 3 at 70% sparsity



LLaMA-7B	Paper	Ours
Dense	5.68	[__]
Wanda	85.77	[__]
OWL + Wanda	24.55	[__]
SparseGPT	26.30	[__]
OWL + SparseGPT	19.49	[__]

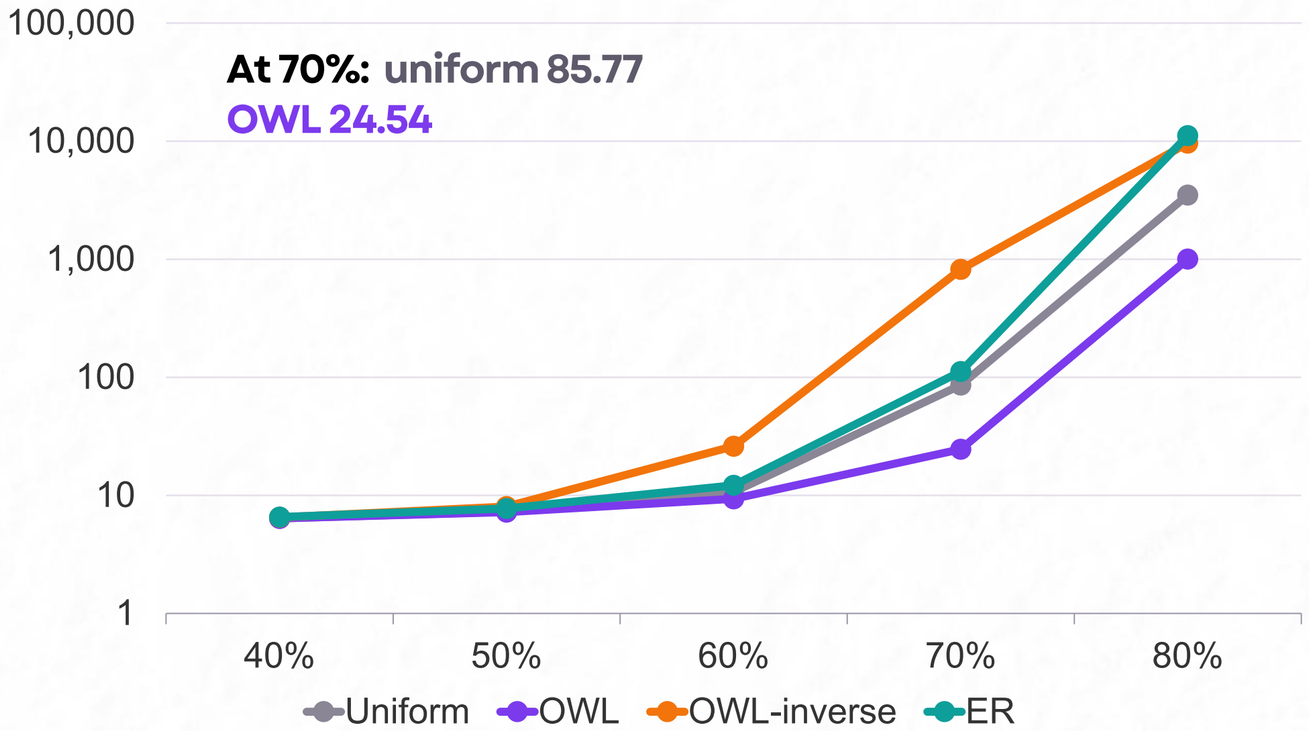
OWL helps in 5 of 6 cases. With SparseGPT on OPT-6.7B it gets worse, and we check that too.

WikiText-2 perplexity, lower is better. Bars show the paper's numbers (Yin et al. 2024, Table 3).

2. Can we reproduce it?

R2 to R4: sweep, zero-shot, outliers

R2: perplexity vs sparsity, LLaMA-7B + Wanda (log scale)



R3: mean zero-shot accuracy, 7 tasks

LLaMA-7B, 70%	Paper	Ours
Dense	62.40	[__]
Wanda	39.39	[__]
OWL + Wanda	46.47	[__]
SparseGPT	44.93	[__]
OWL + SparseGPT	48.03	[__]

R4: outliers per block before and after pruning. We expect the same uneven shape to survive pruning. Ours: [__]

Paper numbers from Yin et al. 2024, Tables 4 and 8. OWL-inverse gives outlier-rich blocks more sparsity.

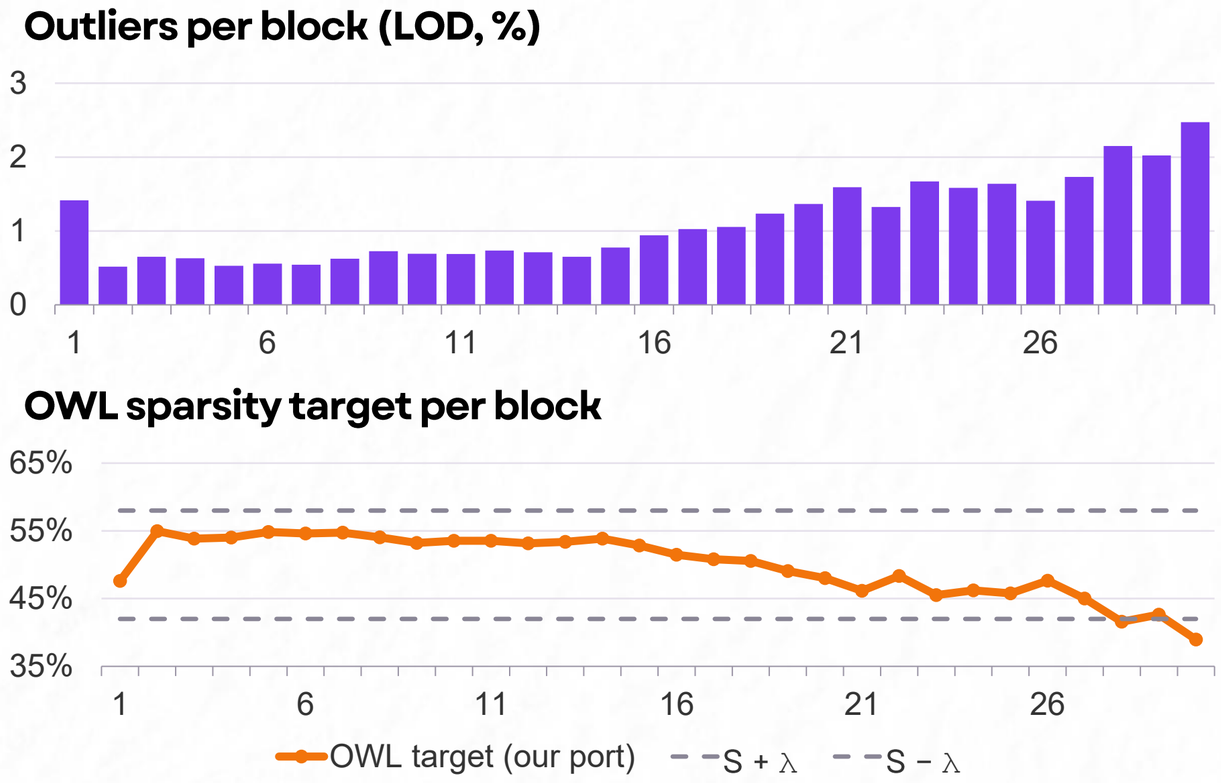
2. Can we reproduce it?

Where we are: code runs, GPU runs next

- ✓ Our port of OWL, Wanda, SparseGPT and magnitude pruning
- ✓ 28 unit tests pass, CPU smoke run works end to end
- ✓ Job lists written for every experiment
- ○ **Next: dense LLaMA-7B check (5.68), then R1 to R4 on GPU**

Found while porting: the released code scales block sparsity into a band 2λ wide, then shifts the mean to S. Some blocks leave the paper's stated range S ± λ.

PyTorch 2.11 🤗 Transformers 5.5

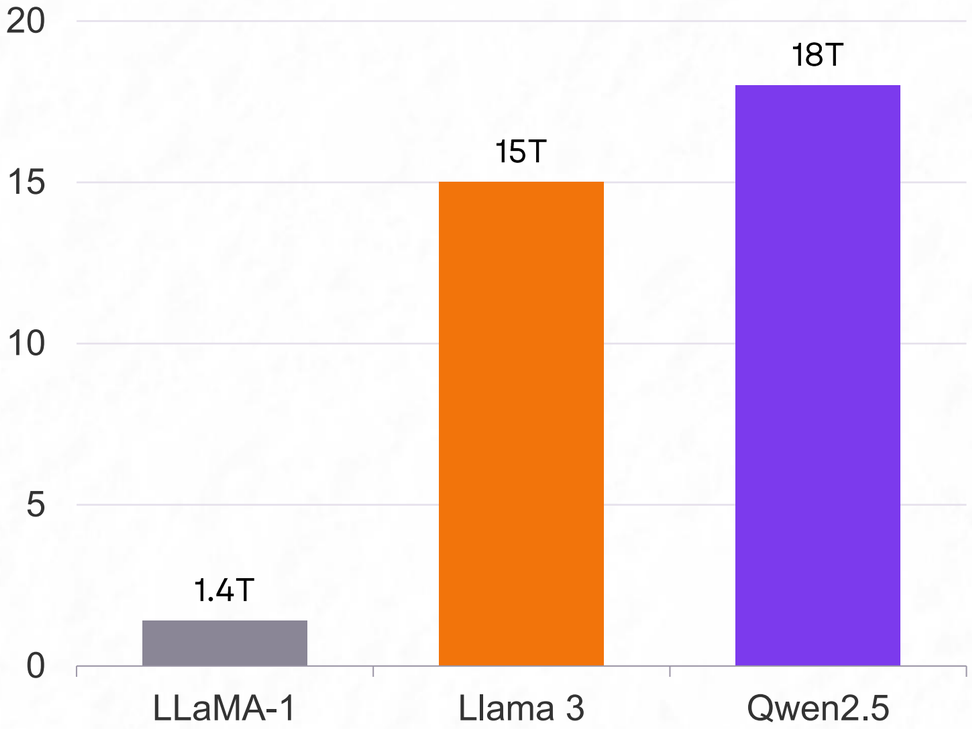


SmolLM2-135M, 50% target, M = 5, λ = 0.08. CPU smoke test with 4 calibration samples, not a result.

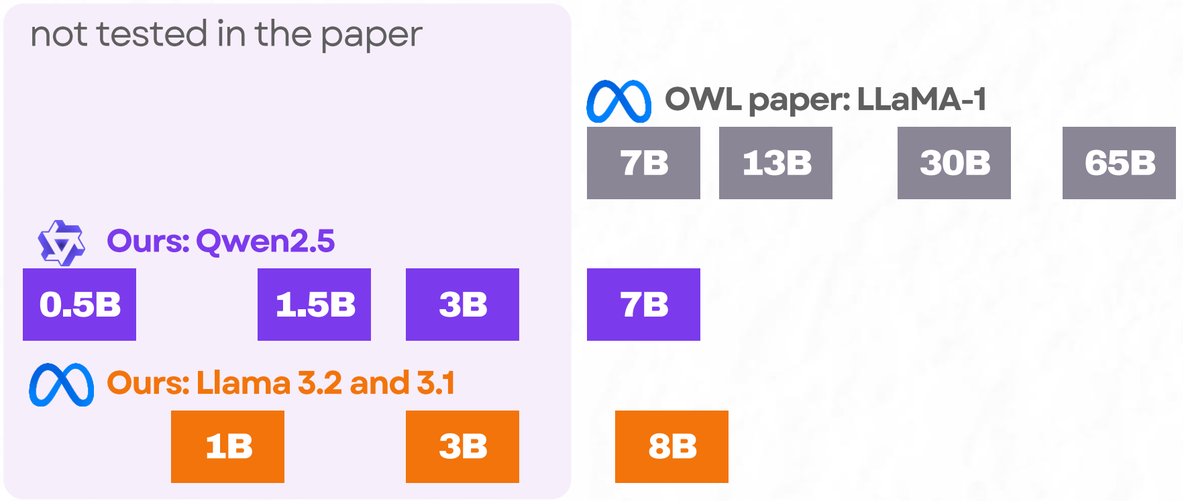
3. Does it hold on new LLMs?

The paper only tested LLaMA-1, 7B and up

Pre-training tokens (trillions)



Model sizes tested (parameters, log scale)



Evidence on modern models is mixed. With SparseGPT at 70%, OWL nearly halves Llama-3-8B perplexity (85.84 to 48.07), but it slightly hurts a 1B Llama 3 at 50%.

Token counts: Touvron et al. 2023, Grattafiori et al. 2024, Qwen Team 2025. Evidence: Sieberling et al. 2025, Choenni et al. 2025.

3. Does it hold on new LLMs?

Three experiments, seven models

E1 **Scaling within a family**



7 models from 0.5B to 8B at 50%, 60% and 70%, uniform vs OWL.

We report the gain in log perplexity and add a shuffled OWL control.

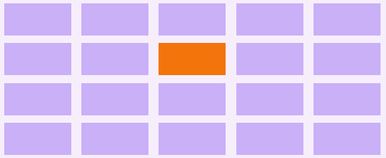
E2 **Does the LOD predict the gain?**



LOD of all 10 dense models: how uneven it is, and whether it forms a U.

Spearman correlation with the OWL gain. Exploratory with 10 models.

E3 **Hyperparameter transfer**



4 x 5 grid
orange: paper default

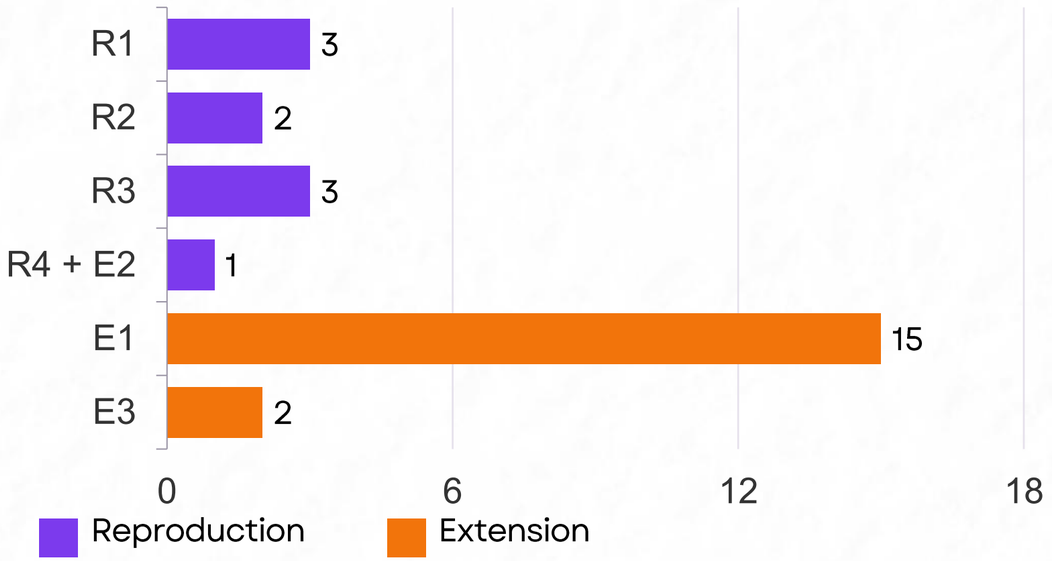
M and λ grid on Qwen2.5-1.5B and Llama-3.2-1B, picked on held-out C4.

A weak result should not just mean bad settings.

4. What do we need?

212 runs, about 26 A100 hours, no paid API

Estimated A100 hours per experiment



212
pruning runs

26 h
estimated, 60 h budgeted

🤗 **$0**
API cost, open weights

- **USF GAIVI cluster** A100, H100 and L40S, free with a supervisor
- **Colab Pro, both accounts** 13 to 37 A100 hours a month
- **Backup** Kaggle, Modal credit or a rented GPU (under $110)
- **GPU memory** 24 GB for 7B and 8B, 31 GB for 13B

Dense check: 5.68 → **R1 to R4** → **E1 to E3** → **Report: Dec 4**

Q&A

Thank you

Code, proposal and slides are in the repo

github.com/khangpt2k6/NLP_CLASS